Built in func, user defined func, return, func call inside another func

1. all()

☞ কাজ: লিস্ট/টাপল/সেটের সব element যদি True হয় তাহলে True রিটার্ন করবে, নাহলে False।

```
numbers = [10, 20, 30, 40]

print(all(num > 0 for num in numbers))   # সবগুলোই 0 এর থেকে বড় => True
print(all(num < 25 for num in numbers))  # সবগুলো 25 এর নিচে না => False
```

Output:

```
True
False
```

---

2. any()

☞ কাজ: লিস্ট/টাপল/সেটে অন্তত **একটা element যদি True হয়** তাহলে True রিটার্ন করবে, না হলে False।

```
numbers = [0, 0, 0, 5]

print(any(numbers))            # অন্তত একটায় True (5) আছে => True
print(any(num < 0 for num in numbers))  # কোনো negative নেই => False
```

🔎 ব্যাখ্যা:

- numbers লিস্টের মানগুলো হলো → [0, 0, 0, 5]
- Python-এ truth value চেক করার নিয়ম:
  - 0, "", None, [], {} ইত্যাদি = False ধরা হয়
  - অন্য সবকিছু (যেমন 5, "abc", [1,2]) = True ধরা হয়

তাহলে এখানে:

- 0 → False
- 0 → False
- 0 → False
- 5 → True

👉 এখন `any(numbers)` মানে হলো:
"লিস্টে অন্তত একটি True আছে কিনা?"

যেহেতু লিস্টে একটা `5` আছে (যেটা `True`), তাই রিটার্ন করবে →`True`

✅ সহজভাবে:

- `all()` = সবগুলো True হতে হবে
- `any()` = অন্তত একটা True হলেই হবে

3. isinstance()

👉 কাজ: কোনো ভেরিয়েবল/অবজেক্ট নির্দিষ্ট টাইপের কি না সেটা চেক করতে।

```
x = 10
y = [1, 2, 3]
z = "Hello"

print(isinstance(x, int))     # True
print(isinstance(y, list))    # True
print(isinstance(z, str))     # True
print(isinstance(z, int))     # False
```

✨ ছোট্ট ট্রিক:
তুমি একসাথে একাধিক টাইপও চেক করতে পারো:

```
print(isinstance(3.14, (int, float)))  # True (কারণ float)
```

1. sorted()

👉 কোনো iterable (list, tuple, set, string ইত্যাদি) কে **ascending বা descending order** এ sort করে **নতুন list** রিটার্ন করে।

```
numbers = [5, 2, 9, 1, 7]

print(sorted(numbers))           # ascending (ছোট থেকে বড়ো)
print(sorted(numbers, reverse=True))  # descending (বড়ো থেকে ছোটো)

text = "python"
print(sorted(text))              # অক্ষরগুলো alphabetically সাজাবে
```

Output:

```
[1, 2, 5, 7, 9]
```

```
[9, 7, 5, 2, 1]
['h', 'n', 'o', 'p', 't', 'y']
```

---

2. reversed()

☞ iterable কে উল্টে দেয়, কিন্তু **নতুন list বানায় না**; বরং একটা iterator দেয়। চাইলে list() দিয়ে list বানানো যায়।

```
numbers = [5, 2, 9, 1, 7]

print(list(reversed(numbers)))   # লিস্ট উল্টে যাবে
print("".join(reversed("python")))   # স্ট্রিং উল্টে যাবে
```

Output:

```
[7, 1, 9, 2, 5]
nohtyp
```

চলো ধাপে ধাপে ব্যাখ্যা করি এই লাইনটা:

```
print("".join(reversed("python")))
```

ধাপ ১: reversed("python")

☞ "python" একটা স্ট্রিং, যেটা character গুলো একটার পর একটা সাজানো আছে:

'p' 'y' 't' 'h' 'o' 'n'

reversed("python") করলে Python উল্টোদিক থেকে character গুলো দেয়:

'n' 'o' 'h' 't' 'y' 'p'

কিন্তু এইটা সরাসরি দেখালে **iterator object** দেখাবে (list না)।

ধাপ ২: "".join(...)

☞ join() ফাংশন character গুলোকে জুড়ে **একটা স্ট্রিং বানায়**।

- "" মানে হলো কোনো ফাঁকা string দিয়ে জোড়া হবে।
- সুতরাং 'n' 'o' 'h' 't' 'y' 'p' → "nohtyp"

ধাপ ৩: `print(...)`

👉 তাই আউটপুট হবে:

```
nohtyp
```

✅ সহজ করে বললে:

- `reversed("python")` → অক্ষর উল্টানো (`['n','o','h','t','y','p']`)
- `"".join(...)` → আবার জুড়ে string বানানো
- তাই `"python"` → `"nohtyp"`

✅ **Key difference:**

- `sorted()` → সবসময় **sort করে** (ascending/descending)।
- `reversed()` → শুধু **উল্টে দেয়**, কোনো sorting করে না।

🔹 1. `lambda` function

👉 এটা হলো **এক লাইনের ছোট function**।
যার নাম থাকে না (anonymous function)।

```
# সাধারণ function
def square(x):
    return x*x

print(square(5))

# lambda দিয়ে
square_lambda = lambda x: x*x
print(square_lambda(5))
```

Output:

```
25
25
```

🔹 2. `map()`

👉 কোনো iterable (list, tuple ইত্যাদি)-এর প্রতিটি element-এর উপর function apply করে।

```
numbers = [1, 2, 3, 4, 5]

squares = list(map(lambda x: x*x, numbers))
```

```
print(squares)
```

Output:

```
[1, 4, 9, 16, 25]
```

⬙ JS এ map উদাহরণ

```
let numbers = [1, 2, 3, 4];
let squares = numbers.map(x => x * x);
console.log(squares);  // [1, 4, 9, 16]
```

- প্রতিটি element x → function x*x apply হয়েছে → নতুন array তৈরি হয়েছে।

⬙ Python এ map ঠিক একইভাবে

```
numbers = [1, 2, 3, 4]

# lambda function ব্যবহার
squares = list(map(lambda x: x*x, numbers))

print(squares)  # [1, 4, 9, 16]
```

- Python এ map **একটা iterator দেয়**, তাই list() দিয়ে কনভার্ট করতে হয় → আমরা দেখতে পাই পুরো result।
- প্রতিটি element x → lambda x: x*x apply হয়েছে → নতুন list তৈরি হয়েছে।

⬙ JS vs Python তুলনা

JS	Python
arr.map(x => x*2)	list(map(lambda x: x*2, arr))
Array রিটার্ন করে	Iterator রিটার্ন করে → list() ব্যবহার করতে হয়
Original array ঠিক থাকে	Original list ঠিক থাকে, নতুন list তৈরি হয়

⬙ 3. filter()

☞ iterable থেকে শুধু সেই elements রাখে যেগুলো **True condition** মেনে চলে।

```
numbers = [1, 2, 3, 4, 5, 6]

even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print(even_numbers)
```

Output:

```
[2, 4, 6]
```

🔹 4. reduce()

👉 সব element কে একত্রে **একটা মানে নামিয়ে আনে** (accumulation করে)।
⚠ reduce ব্যবহার করতে হলে functools থেকে import করতে হয়।

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]

sum_of_numbers = reduce(lambda x, y: x + y, numbers)
print(sum_of_numbers)
```

Output:

```
15
```

(কারণ: (((1+2)+3)+4)+5 = 15)

✅ সহজভাবে মনে রাখার ট্রিক:

- map() → সব element কে বদলে দেয়
- filter() → কিছু element বেছে রাখে
- reduce() → সব element মিলে এক মানে নামায়
- lambda → এক লাইনের ছোট function

◆ 1. Function কী?

Python এ **function** হলো কোডের একটি **block** যা কোনো কাজ সম্পন্ন করে এবং প্রয়োজন হলে **value return** করে।

- Function কোডকে **reuseable** করে।
- বড় প্রোগ্রাম structure করার জন্য function খুবই গুরুত্বপূর্ণ।

◆ 2. Function বানানোর syntax

```
def function_name(parameters):
    """
    Optional docstring - function কী করবে সেটা লিখতে পারো
    """
    # কোড
    return value  # optional
```

🔹 3. Example 1: Simple Function

```
def greet(name):
    print(f"Hello, {name}!")

greet("Nusrat")  # Hello, Nusrat!
```

🔹 4. Example 2: Function with return value

```
def add(a, b):
    return a + b

result = add(5, 3)
print(result)  # 8
```

🔹 5. Function Parameters

1) Positional Arguments

```
def greet(first, last):
    print(f"Hello, {first} {last}!")

greet("Nusrat", "Korayil")
```

2)Keyword Arguments

```
greet(last="Korayil", first="Nusrat")
```

3) Default Parameters

```
def greet(name="Friend"):
    print(f"Hello, {name}!")

greet()          # Hello, Friend!
greet("Nusrat")  # Hello, Nusrat!
```

4)Arbitrary Arguments (*args & **kwargs)

```
def add_numbers(*args):
    return sum(args)

print(add_numbers(1,2,3,4))  # 10

def show_info(**kwargs):
    print(kwargs)

show_info(name="Nusrat", age=28)
# {'name': 'Nusrat', 'age': 28}
```

⬙ 6. Function Scope

- Function এর ভেতরের variable হলো **local**।
- Function এর বাইরে access করা যায় না।

```
def my_func():
    x = 10  # local variable
    print(x)

my_func()
# print(x) → error, x undefined
```

⬙ 7. Lambda Function (Short Function)

Syntax:

lambda arguments:expression

```
square = lambda x: x*x
print(square(5))  # 25
```

✅ সংক্ষেপে:

Function Feature	Example
Positional Arguments	`def add(a,b)`
Keyword Arguments	`add(b=2, a=3)`
Default Parameter	`def greet(name="Friend")`
Arbitrary Arguments	`*args, **kwargs`
Return Value	`return value`
Lambda (Anonymous)	`lambda x: x*x`

return:

◆ 1. return কী করে?

👉 return function থেকে বেরিয়ে এসে কোনো **value (result)** কলারের কাছে পাঠিয়ে দেয়।

```
def add(a, b):
    return a + b

result = add(5, 3)
print(result)   # 8
```

◆ 2. Function without `return`

☞ যদি function এ return না থাকে, তাহলে ডিফল্ট হিসেবে **None** ফেরত দেয়।

```
def greet(name):
    print(f"Hello, {name}!")

x = greet("Nusrat")
print(x)   # None
```

◆ 3. Single Value Return

```
def square(x):
    return x * x

print(square(4))  # 16
```

◆ 4. Multiple Values Return

☞ Python function একসাথে একাধিক value ফেরত দিতে পারে (tuple আকারে)।

```
def get_values():
    return 10, 20, 30

a, b, c = get_values()
print(a, b, c)   # (10 20 30)
```

◆ 5. Returning Expressions

☞ return সরাসরি expression evaluate করে value দেয়।

```
def bigger(a, b):
    return a if a > b else b

print(bigger(7, 4))  # 7
```

◆ 6. Early Return

☞ function এর মধ্যে যেকোনো সময় return করলে function সাথে সাথে শেষ হয়ে যায়।

```
def check_number(n):
    if n < 0:
        return "Negative"
    return "Positive or Zero"

print(check_number(-5))   # Negative
print(check_number(10))   # Positive or Zero
```

◆ 7. Return None Explicitly

```
def do_nothing():
    return None

print(do_nothing())  # None
```

- **`return None`** = function খালি হাতে ফিরে আসবে।
- যদি function এ **কোনো return না লেখো**, তাহলেও Python স্বয়ংক্রিয়ভাবে None ফেরত দেয়।

উদাহরণ:

```
def do_nothing2():
    pass    # কিছু করছে না

print(do_nothing2())    # Output: None
```

☞ মানে, None হচ্ছে Python এর একটা বিশেষ মান, যেটা "কিছু নেই" বা "খালি" বোঝায়।

◆ 8. Return in Loops

```
def find_first_even(numbers):
    for n in numbers:
        if n % 2 == 0:
            return n   # প্রথম even পেলেই return করবে
```

```
    return None        # না পেলে None ফেরত

print(find_first_even([1, 3, 5, 8, 9]))  # 8
```

✅ **সংক্ষেপে Return এর কাজ:**

- Function থেকে result caller এর কাছে পাঠানো
- return এর পর কোড আর চলে না
- একাধিক value ফেরত দেওয়া যায় (tuple আকারে)
- যদি কিছু return না করা হয় → ডিফল্ট None

◆ Python Function এ return করার বিভিন্ন ধরন

1)কোনো return না দেওয়া

👉 Function শেষে কিছু লেখা না থাকলে বা শুধু pass থাকলে ডিফল্ট হিসেবে None ফেরত দেয়।

```
def func1():
    pass

print(func1())    # None
```

---

2)Explicit return None

👉 সরাসরি None রিটার্ন করা।

```
def func2():
    return None

print(func2())    # None
```

---

3)একটা single value return করা

👉 সাধারণত সংখ্যার result বা string ফেরত দেওয়ার সময় ব্যবহার হয়।

```
def square(x):
    return x * x

print(square(5))  # 25
```

---

4)Multiple values return করা (tuple আকারে)

☞ Python function একসাথে একাধিক value ফেরত দিতে পারে।

```
def info():
    return "Nusrat", 28, "Bangladesh"

name, age, country = info()
print(name, age, country)
# Nusrat 28 Bangladesh
```

---

5) Expression return করা

☞ Condition বা formula থেকে সরাসরি expression return করা যায়।

```
def bigger(a, b):
    return a if a > b else b

print(bigger(10, 7))  # 10
```

---

6)Return inside loop (early return)

☞ Loop এর মধ্যে কোনো condition মিলে গেলে সঙ্গে সঙ্গে return করে function শেষ হয়ে যায়।

```
def find_even(nums):
    for n in nums:
        if n % 2 == 0:
            return n   # প্রথম even number পেলেই বের হবে
    return None

print(find_even([1, 3, 7, 10, 15]))  # 10
```

---

7) Return list / dictionary / object

☞ function চাইলে সরাসরি data structure ফেরত দিতে পারে।

```
def get_list():
    return [1, 2, 3, 4]

def get_dict():
    return {"name": "Nusrat", "age": 28}

print(get_list())   # [1, 2, 3, 4]
print(get_dict())   # {'name': 'Nusrat', 'age': 28}
```

---

8)Return function itself (higher-order function)

☞ Function থেকে অন্য function ফেরত দেওয়া যায়।

```
def outer():
    def inner():
        return "Hello from inner!"
    return inner

func = outer()
print(func())  # Hello from inner!
```

✅ সংক্ষেপে return করার উপায়

1. Return না দিলে → `None`
2. Explicit `return None`
3. Single value return
4. Multiple value (tuple) return
5. Expression return
6. Early return (loop/condition এর ভেতর থেকে)
7. Data structure return (list, dict, object)
8. অন্য function return করা

Others

```
def user_input():
    x1=int(input())
    x2=int(input())
    x3=int(input())
    x4=int(input())
    return (x1,x2),(x3,x4)
r1,_=user_input()
print(r1)
#input: 1,2,3,4
```

#output: (1,2). _আরেকটা রিটার্ণ ভ্যালুকে অমিট করে

◆ 1. Simple Example: Function call inside another function

```
def greet(name):
    return f"Hello, {name}!"

def welcome_message(name):
    message = greet(name)   # greet function কে call করা হলো
    return message + " Welcome to Python."

print(welcome_message("Nusrat"))
```

Output:

```
Hello, Nusrat! Welcome to Python.
```

✅ এখানে আমরা `greet()` function কে `welcome_message()` function-এর ভিতরে ব্যবহার করেছি।

◆ 2. Using return value from one function in another

```
def square(x):
    return x * x

def sum_of_squares(a, b):
    return square(a) + square(b)

print(sum_of_squares(3, 4))  # 3^2 + 4^2 = 9 + 16 = 25
```

- `sum_of_squares()` এর মধ্যে `square()` call করা হয়েছে
- `square()` return value সরাসরি ব্যবহার হয়েছে

◆ 3. Nested function call (function inside function)

```
def outer(x):
    def inner(y):
        return y + 5
    return inner(x) * 2

print(outer(10))  # (10+5)*2 = 30
```

- `inner()` function শুধু `outer()` এর ভিতরে আছে
- শুধু `outer()` call করলে inner কাজ করে

◆ 4. Function as argument to another function

```
def multiply_by_2(x):
    return x * 2

def apply_func(func, value):
    return func(value)

print(apply_func(multiply_by_2, 5))  # 10
```

- এখানে function `multiply_by_2` অন্য function `apply_func` কে argument হিসেবে দেওয়া হলো
- Python এ functions **first-class objects**, তাই এটা possible

---

◆ Summary / Key Points

1. এক function থেকে অন্য function call করা যায় `function_name()` দিয়ে
2. return value সরাসরি অন্য function এ ব্যবহার করা যায়
3. function ভিতরে function define করা যায় (nested)
4. function কে argument হিসেবে আরেকটি function এ পাঠানো যায়